T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge

Jianyu Wei*
USTC / Microsoft Research
China
noob@mail.ustc.edu.cn

Shijie Cao†
Microsoft Research
shijiecao@microsoft.com

Ting Cao†
Microsoft Research
ting.cao@microsoft.com

Lingxiao Ma
Microsoft Research
lingxiao.ma@microsoft.com

Lei Wang*
UCAS / Microsoft Research
t-leiwang@microsoft.com

Yanyong Zhang
USTC
yanyongz@ustc.edu.cn

Mao Yang
Microsoft Research
maoyang@microsoft.com

arXiv:2407.00088v2 [cs.DC] 25 Mar 2025

Abstract

The deployment of Large Language Models (LLMs) on edge devices is increasingly important to enhance on-device intelligence. Weight quantization is crucial for reducing the memory footprint of LLMs on devices. However, low-bit LLMs necessitate **mixed precision matrix multiplication (mpGEMM)** of low precision weights and high precision activations during inference. Existing systems, lacking native support for mpGEMM, resort to dequantize weights for high precision computation. Such an indirect way can lead to a significant inference overhead.

In this paper, we introduce T-MAC, an innovative lookup table(LUT)-based method designed for efficient low-bit LLM (i.e., weight-quantized LLM) inference on CPUs. T-MAC directly supports mpGEMM without dequantization, while simultaneously eliminating multiplications and reducing additions required. Specifically, T-MAC transforms the traditional data-type-centric multiplication to bit-wise table lookup, and enables a unified and scalable mpGEMM solution.

Our LUT-based kernels scale linearly to the weight bit-width. Evaluated on low-bit Llama and BitNet models, T-MAC demonstrates up to 4× increase in throughput and 70% reduction in energy consumption compared to *llama.cpp*. For BitNet-b1.58-3B, T-MAC delivers a token generation throughput of 30 tokens/s with a single core and 71 tokens/s with eight cores on M2-Ultra, and 11 tokens/s on Raspberry Pi 5. T-MAC with LUT-based computing paradigm, paves the way for the practical deployment of low-bit LLMs on resource-constrained edge devices without compromising computational efficiency. The system is open-sourced at https://github.com/microsoft/T-MAC.

ACM Reference Format:
Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Twentieth European Conference on Computer Systems (EuroSys '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3689031.3696099

*Work done during the internship at Microsoft Research.
†Corresponding Author.

1 Introduction

More and more large language models (LLMs) are deploying on client devices, such as smartphones, desktops, and robotics, to provide unprecedented intelligence services, real-time task response and user data protection. Typical examples are Phi-3-mini-4bit deployed on iPhone [12], Llama-2-7B-4bit on Pixel 5 and Llama-2-13B-4bit on Apple M2 Ultra [5], as well as the recent Microsoft Copilot+PC [3] that collaboratively run the on-device LLM with the on-cloud LLMs.

Low-bit weight quantization is a must-have for on-device LLM inference due to the hardware resource limitation. LLM inference quality is also robust to precision loss. Beyond 4-bit, 3-bit, 2-bit and even 1-bit models are emerging [14, 19, 34, 36]. By comparison, activation quantization cannot follow the trend due to outliers. The computing operands thus have asymmetric precision and bit width, such as W4A16, W2A16, or W1A8[1].

On the other hand, current commodity hardware still supports fixed bit-width and symmetric operands, and cannot support these various and mixed precision. Even some research papers support asymmetric bit-width operands. The bit-width is still fixed, such as W4A8 [23]. Computing kernels have to convert/dequantize low-bit weight to match the activation, and compute on hardware.

This conversion raises two obvious issues. *(i)* Performance wise, the conversion cost offsets the gain from bit scaling down. Our evaluation (ref Figure 6) shows that scaling down bits from 4 bit to 1 bit even increases latency cost for most of the cases. *(ii)* Development wise, the data layout and kernels need to be designed case by case for each mixed precision. For example, the data layouts, as well as the interleaving or swizzling methods for W3 and W2 are totally different. The kernels need to be redesigned to match the layout. Therefore, to deploy LLM on devices, a fundamental problem is how to directly and efficiently support mpGEMM (mixed precision General Matrix Multiplication) of low bit weights and high bit activations on devices.

This paper aims a mpGEMM kernel design, which is independent of hardware data types and the bit width of the

[1]W# means the bit-width of weight and A# means the bit-width of activation

quantization algorithms, to achieve scalable speedup with bit scaling down. To realize that, our *key concept* is that rather than the dominant data-type-centric calculation, we exploit the *bit-wise* calculation of the standard algorithm of multiplication. That is, the multiplication of two numbers can be transformed into multiplying one number by each bit of the other number, then shifting and adding the partial products. The mpGEMM between activation and weight matrices is decomposed into a series (= the bit-width of weight) of mpGEMM between activation and a one-bit matrix, and then adds the partial results up. This method can thus support any bit-width combination of activation and weight.

A promising method to implement bit-wise calculation is by table lookups [30]. Since one bit can only represent two values, e.g., 1/-1, the bit patterns of a one-bit vector are limited. For example, if a one-bit matrix is partitioned into groups of four-element vector, the number of possible bit patterns (e.g., [1,1,1,-1] and [1,1,-1,-1]) for each group is only 2^4. Given an activation, it can be first computed with all possible bit patterns and saved in tables. The mpGEMM of activation and one-bit matrix is then transformed to table lookup indexed by each bit pattern in weight, and addition to accumulate the looked-up results. The mpGEMM is reduced to table lookup+add operations and no multiplication.

Though bit-wise LUT-based mpGEMM could reduce multiplications, how to efficiently implement it on real devices is challenging, since current hardware is highly tailored to multiplication. The *outstanding difference* between bit-wise LUT and traditional mpGEMM is the two operands for LUT method are tables and index matrices, rather than activation and weight. The data format and layout of the two operands are thus critical to the inference speed. *(i)* One challenge is that compared to the continuous data access for activation and weight, the access to tables are random. The residence of tables in fast on-chip memory will be particularly important for final inference performance. *(ii)* However, the on-chip memory is limited, and the LUT method enlarges the on-chip memory usage compared to traditional mpGEMM. This is because the LUT needs to save the results of the activation vector multiplied with all possible bit patterns. This is exponentially more than the activation itself.

By solving the challenges, this paper proposes T-MAC mpGEMM kernel library. As shown in Figure 1, it is based on the concept of bit-wise calculation and LUT implementation, and provide unified and scalable solution for any mixed bit-width of activation and weight. To alleviate the random-access cost of LUT, we propose techniques in both system and algorithm to enable the LUT to reside in the fastest on-chip memory and parallel lookup. On system-side, we propose the *LUT-centric data layout*, to make sure a LUT in on-chip memory i.e., registers, through axis reordering and tiling to fully reuse each table as well as reduce the temporary results

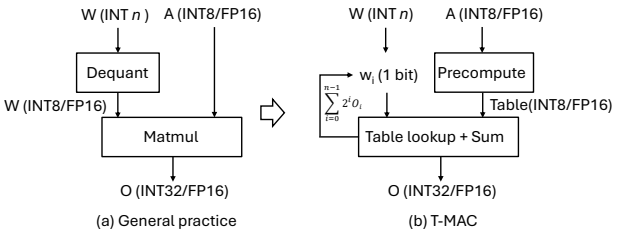

Figure 1. T-MAC vs general practice for mpGEMM.

which competes with the on-chip memory. On algorithm-side, we propose *table quantization* and *mirror consolidation* to reduce the size of tables.

We implemented T-MAC on the pervasively available CPU processors of edge devices, even a Raspberry Pi. We find T-MAC makes the LLM inference speed on a CPU comparable or even higher to the GPU on the same device, mainly because T-MAC has no conversion cost and the total operation reduction by table lookup. *This paper is thus the first to provide a practical solution for deploying LLMs on edge devices using the widely available CPUs, without relying on GPUs.* We evaluated T-MAC performance on typical edge devices, including Apple M2 Ultra, Jetson AGX Orin, Surface Book 3, and Raspberry Pi 5. The T-MAC kernel speedup can reach up to 6.6× and an average of 3.6× compared to the SOTA on the CPU by llama.cpp [5]. The e2e LLM inference speedup achieves 2.8× speedup for Llama-2-7B-2bit model [4]. The inference performance can reach 11.1 tokens/s even on a Raspberry Pi for BitNet-b1.58-3B model [34]. T-MAC is also energy efficient, reducing 60-70% energy compared to llama.cpp.

Our contributions can be summarized as follows:

- T-MAC transforms the data-type-centric multiplication to *bit-wise* table lookup for a unified and scalable mpGEMM design.
- We propose both system and algorithm techniques to enable the table to reside on the fastest memory and parallel lookup.
- We implement the T-MAC kernel library and e2e inference system to achieve significant LLM inference speedup and energy saving.

2 Background and Motivation

2.1 LLM on Edge

The advent of Large Language Models (LLMs) has revolutionized the field of natural language processing, opening new horizons for human-computer interaction and personalized assistant. The deployment of LLMs directly on edge devices, such as smartphones, desktop computers, and robotics, has emerged as a critical frontier in computing, promising to enhance these devices with unprecedented levels of intelligence and autonomy.

LLM-on-edge benefits. Deploying LLMs on edge devices brings several compelling benefits. On-device processing drastically reduces response latencies, which is critical for

time-sensitive applications such as autonomous vehicles and interactive robotics. Furthermore, local data processing enhances user privacy by keeping sensitive information confined to the device, thereby reducing the risk of data leakage. Another significant advantage is the operational reliability that comes with network independence, allowing for consistent functionality regardless of network availability or stability.

LLM-on-edge challenges. The primary challenge in deploying LLMs on edge devices is the substantial memory requirement to accomodate these models. LLMs often encompass billions of parameters. For example, LLAMA-2-7B with FP16 precision requires at least 14GB of memory to host the model. In contrast, edge devices typically have limited memory resources, which poses a stark limitation for on-device deployment.

Beyond memory capacity, the computational and memory bandwidth demands of LLMs present a significant hurdle for edge deployment. Edge devices often process data in a single-instance fashion, commonly with a batch size of one, to cater to a single user's real-time interactions. The inference process of the LLM can be divided into two stages: the prefill and decode. The prefill stage, with the self-attention mechanism applied across all input tokens, involves compute-intensive matrix-matrix multiplications. However, once the key-value (KV) cache is generated, the decode stage becomes the bottleneck. In this phase, generating each subsequent token necessitates laoding and processing the entire model, which translates to memory-intensive matrix-vector multiplications.

The third challenge is power or energy efficiency, which is particularly critical for battery-operated edge devices such as smartphones and robotics. These devices are designed for prolonged operation on finite energy reserves, making energy efficiency an essential consideration.

2.2 Low-Bit (Weight-Quantized) LLM

The memory-intensive nature of LLM inference necessitates strategies to reduce the model's memory footprint without significantly compromising model performance. Weight quantization emerges as a key technique to achieve this balance.

Weight quantization involves reducing the precision of the model's parameters, effectively allowing the model to occupy less memory and potentially speed up computation by leveraging lower-precision arithmetic [18, 21, 26]. Nowadays, LLMs are increasingly being released with a 4-bit version specifically tailored for deployment on edge or other resource-constrained environments [33, 37]. Recent research has pushed the boundaries even further, investigating the feasibility of 2-bit and 1-bit weight representations in LLMs [19, 34, 36]. Fundamentally, the choice of bitwidth or precision in weight quantization represents a trade-off between computational efficiency and model accuracy.

2.3 Deployment Challenges of Low-Bit LLM

The adoption of low-bit LLMs has become a necessity for edge deployment. In fact, numerous LLM-on-Edge systems and implementations are actively employing low-bit techniques [2, 5]. However, deploying low-bit LLMs introduces unique computational challenges particularly in accommodating mixed-precision operations, which are not natively supported by most hardware architectures, and managing the diversity of bit widths and precision levels required by various deployment scenarios. Addressing these challenges is critical for full potential and seamless intergration of low-bit LLMs in edge computing.

Mixed-precison GEMM/GEMV. The utilization of low-bit (weight-quantized) LLMs introduces a computational paradigm where low precision weights are combined with relatively higher precision activations. This necessitates a shift from standard matrix multiplication operations (i.e., GEMM, GEMV) to mixed-precision ones (i.e., mpGEMM, mpGEMV). However, current hardware architectures, including CPUs, GPUs, and NPUs, do not natively support mixed-precision operations. These architectures are traditionally optimized for standard operations where both operands share the same data type and precision level.

In response to this limitation, existing systems implement an indirect approach by employing dequantization, which involves converting low-precision weights back to a higher precision to align with activation precision. This process enables the use of high-precision GEMM for low-bit LLM inference. For instance, systems like those used in the *Intel Neural Compressor* and *llama.cpp* rely on this dequantization-based technique. However, the efficacy of such methods is contingent upon the assumption that dequantization does not become a bottleneck and can be overlapped with memory loading. Since this indirect approach ultimately reverts to high-precision computation, it fails to fully capitalize on the benefits of low-bit weights, such as reduced memory usage and potentially faster computation.

Bit-width/precision diversity. Beyond the challenge of facilitating mixed-precision operations, the diversity of bit-widths and precisions required by different deployment scenarios compounds the complexity. Depending on the task's difficulty and the specific requirements of the deployment environment, a variety of bit-widths may be selected to optimize performance. No single bit-width or precision setting can universally satisfy the diverse demands of all possible use cases. Consequently, this necessitates computational approaches capable of supporting a spectrum of low-bit widths, ensuring adaptability to the wide-ranging needs of edge computing tasks.

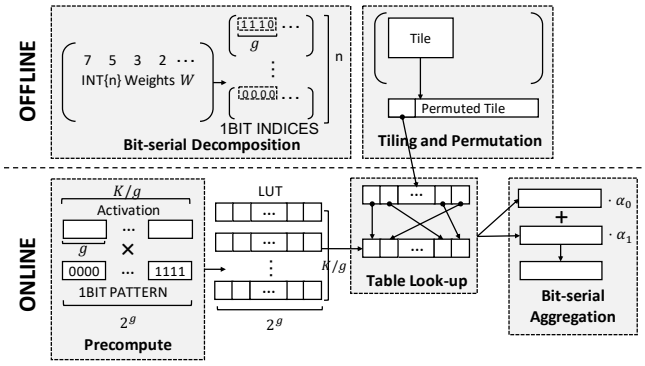

Figure 2. T-MAC design overview.

2.4 LUT-based Computation for Quantized Model

A new trend in the computation of quantized models is the adoption of Lookup Table (LUT)-based methods. For quantized Convolutional Neural Networks (CNNs), where both weights and activations are quantized to levels such as 4-bit, 2-bit, or 1-bit, DeepGEMM [22] precomputes all possible products of weights and activations, stores them in a lookup table, and efficiently accesses them at inference time to avoid costly multiply-accumulate operations. Another example is for vector quantization, where activations are vector quantized, MADDNESS [13] and LUT-NN [32] also transform GEMM computation to table lookups.

In the context of low-bit LLMs, or weight-only quantized LLMs, the LUT-based approach has been explored on GPUs [27, 30]. These methods leverage the GPU's shared memory or cache to store and access the lookup tables. However, despite the theoretical reduction in computational complexity, the practical kernel performance is worse than dequantization-based kernels in [7, 8]. For example, when tested with weight matrix shapes from real-world Llama-2 models on A100 GPU, the average latency of LUT-GEMM kernels [30] is 2.34×, 1.87×, and 1.75× longer than dequantization-based kernels in BitBLAS [7] for $W_{\text{INT4}}A_{\text{FP16}}$, $W_{\text{INT2}}A_{\text{FP16}}$, and $W_{\text{INT1}}A_{\text{FP16}}$ mpGEMVs, respectively. The suboptimal kernel performance is attributable to the constraints of the GPU's fixed architecture, which offers either inadequate storage capacity for the lookup tables or insufficiently rapid table access. In contrast, the exploration of LUT-based mixed-precision GEMM/GEMV on CPUs remains uncharted. Our work pioneers this investigation by examining the viability and performance implications of applying LUT-based methods to CPU-based inference of low-bit LLMs.

3 Design

Current implementations for mixed-precision GEMM vary case by case. Each bit-width combination of activation and weight, such as W4A16 and W2A8, requires specific weight layout and computing kernels. For example, the layout for W3 could pack 2 bits and the other 1 bit in separate, and leverages different interleaving or swizzling methods for memory alignment or fast decoding. The corresponding computing kernel then needs to unpack this specific layout to a hardware-supported data type for execution.

To provide a unified and scalable solution for mixed-precision GEMM, this paper transforms the dominant *data-type-centric* computation to *bit-wise* computation, based on the linear equivalent transformation in Eq. 1. For mixed-precision GEMM, A and W are the activation and weight matrix, respectively. n is the bit-width of the weight. W_i is each bit matrix of W.

$$A \times W = A \times (\sum_{i=0}^{n-1} 2^i W_i) = \sum_{i=0}^{n-1} 2^i A \times W_i \tag{1}$$

In this way, the diverse weight layouts are reduced to a unified one-bit matrix layout. The diverse computing kernels are reduced to unified multiplication of the activation matrix and the one-bit matrix. Besides, bit-wise computation enables the linear scale-down of computation cost with the bit-width reduction.

This paper exploits the LUT method to realize this bit-wise layout and multiplication (Sec. 3.1), and proposes the LUT-centric data layout (Sec. 3.2), as well as the table compression methods (Sec. 3.3) to enable in-register table residence and the fastest parallel lookup.

3.1 T-MAC Algorithm

Figure 2 and and Alg. 1 shows the T-MAC design. During the offline preparation stage (line 29 to 35), a n-bit weight matrix is decomposed into n one-bit matrices. Since one bit can only represent two values, for a group with g bits, the possible permutations are only 2^g. During the online stage, the permutations can be precomputed with each group of activation of shape $[1, g]$ and saved in a table. A g-bit group in the weight is thus an index to look up the table for the precomputed results. Therefore, a *table* in T-MAC is defined to save the results of a $[1, g] \times [g, 2^g]$ sub-matrix multiplication, and the table size is $[1, 2^g]$. During the offline stage, a tile in the one-bit matrix will be saved continuously in memory to facilitate fast loading. Same as the tiling of normal matrix multiplication, a tiling here is also to improve data locality and cache utilization during LUT.

For the online stage, given the input activation of a GEMM, T-MAC traverses every $[1, g]$ vector of the activation to multiply with the $[g, 2^g]$ bit-pattern matrix and build up a table (line 16 to 27). During LUT, each index (i.e., group) of the one-bit weight matrix is used to look up the tables for partial results (line 6-9). The accumulation of the partial results will be the final GEMM results (line 12-14).

To illustrate with an example, given g=4, for an activation (A1,A2,A3,A4) of shape $[1, 4]$ and 1-bit weights of shape $[4, M]$, the activation will be precomputed online into a LUT of shape $[1, 16]$, containing elements from -A1-A2-A3-A4 to A1+A2+A3+A4. By grouping every 4 weights together, the weight vector of 0000 will lookup -A1-A2-A3-A4 and 0101 will lookup -A1+A2-A3+A4.

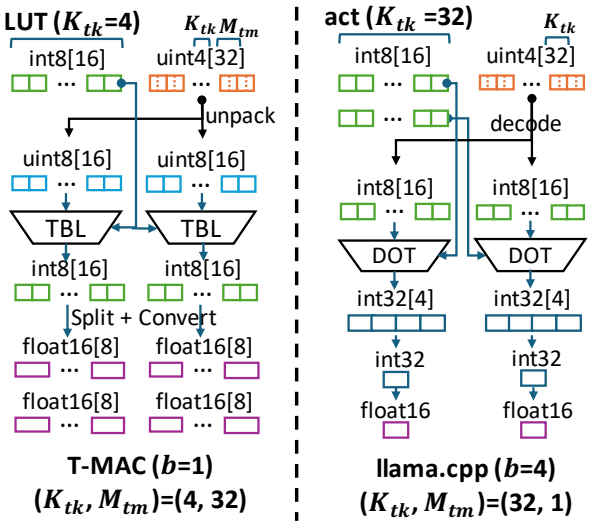

Figure 3. Data flow of T-MAC vs general practice.

An example of LUT-based mpGEMM. Figure 3 (left) shows an example of the T-MAC materialization on CPU. Taking the group size $g = 4$, the tile size of the index matrix $W_i[K_{tk}, M_{tm}] = [4, 32]$, and the bit width as $b = 4$. The right side is the general practice of mpGEMM implementation using llama.cpp. For T-MAC on the left side, the 32 uint4 indices are first unpacked into uint8 bytes (blue) to ensure compatibility with the hardware data type and instructions. Subsequently, the uint8 indices are utilized to look up the table. The results from the look-up are then split and converted to a higher precision for multiplying with the quantization scales of the low-bit LLM model.

By comparison, the general practice designs specific computing kernels for this 4-bit model. It first directly decodes the 4-bit weight to int8 to align with hardware data type, and then conducts int8 dot-product for the activation and weight vectors. Similarly, the result will be covered into FP16 for multiplying with quantization scales. The cache tiling for llama.cpp is $W[K_{tk}, M_{tm}] = [32, 1]$. Sec. 3.2 will explain the different tiling rationale of T-MAC from current practice.

Challenges of LUT implementation. From the algorithm and example, it can be seen that LUT-based mpGEMM exposes the following challenges. *(i) Random data access.* Compared to the continuous data access of current practice, the tables are randomly accessed given the indices. It is necessary to host tables in the fast on-chip memory to reduce the access cost. *(ii) Enlarged on-chip memory usage.* However, LUT requires more on-chip memory compared to current practice. The table size grows exponentially with the group size g. For instance, when $g = 4$, the LUT is four times larger than the original activation. Besides, in contrast to the traditional GEMM implementation that yields *scalar* outputs for each basic block, LUT method results in *vector* outputs (as shown in Figure 3), which requires more on-chip memory to store the temporary results. Considering the example in Figure 3 again, the LUT method uses 144 8-bit registers and llama.cpp uses 104 8-bit registers.

To address the above issues, we propose two main techniques: (a) *LUT-centric data layout* to accommodate intermediates and LUT into memory with higher bandwidth, and (b) *reduced LUT storage* to decrease the LUT size and limit the number of look-up operations.

Algorithm 1: T-MAC GEMM

input : Activation A of shape N, K,
weights W of shape M, K
output : Result matrix R of shape N, M

let b be the number of bits in weights
$W_1, ..., W_b \leftarrow$ PreprocessWeights(W, M, K)
LUT $\leftarrow$ Precompute(A, N, K)
let α_i ($i \leftarrow 1$ to b) be the multiplier of bit serial
let β be the bias of bit-serial
for $i \leftarrow$ *1 to* b **do**
  for $n, m \leftarrow$ *1 to* N, M **do**
    $R_i[n, m] = \sum_{k \leftarrow 1}^{K}$ Look-up(LUT, W_i, n, m, k)

let B be a matrix of shape M, K with all elements β
$R_\beta \leftarrow A \cdot B^T$
$R \leftarrow \sum_{i \leftarrow 1}^{b} \alpha_i R_i + R_\beta$

Function `Precompute`(A, N, K):
  let g be the group size of LUT table
  for $n, k \leftarrow$ *1 to* N, K **do**
    for $i, j \leftarrow$ *1 to* $2^g, g$ **do**
      if i & $(1 \ll j)$ **then**
        LUT[$n, k/g, i$] += $A[n, k]$
      else
        LUT[$n, k/g, i$] -= $A[n, k]$
  return LUT

Function `PreprocessWeights`(W, M, K):
  for $i \leftarrow$ *0 to* b **do**
    for $m, k \leftarrow$ *1 to* M, K **do**
      $W_i[m, k/g]$ += (W[m, k] » i) « (k % g)
  return $W_1, ..., W_b$

3.2 LUT-Centric Data Layout

As described in §3.1, the LUT-based method for low-bit GEMM requires more memory to store the lookup table and the intermediate results. On the other hand, the table look-ups usually lead to memory access inefficiency due to the random memory accesses. To resolve the inefficiency on memory storage and accesses, we design a LUT-centric data layout for the LUT-based low-bit GEMM. Specifically, it stores the lookup table on on-chip memory like registers

to accelerate table accesses, and designs axis reordering and data tiling to enhance data reuses for reducing memory consumption. Furthermore, to improve the efficiency, we design two data layout optimizations, i.e., weight permutation to align with memory transactions, and weight interleaving for optimizing weight unpack.

Put lookup table on on-chip memory. The LUT-based GEMM described in §3.1 requires fetching the result from the pre-computed table, which should access the lookup table randomly. To accelerate these table look-ups, we put the lookup table on registers, and leverage the hardware-specific instructions (e.g., TBL on ARM CPUs and PSHUF on x86 CPUs) to do table look-ups. The details of optimizing table look-ups with hardware-specific instructions will be described in §4. However, putting the lookup table on registers further increases the memory pressure on the on-chip memory, which may result in dramatic performance drop due to memory spilling. To fully explore the potential of on-chip memory table look-ups, we design axis reordering and tiling to enhance data reuses to reduce the on-chip memory pressure.

Axis reordering. For GEMM $C[N,M] = A[N,K] \times W[M,K]$, it is natural to loop among the spatial axes N and M, and then the temporal axis K. However, the LUT-based GEMM needs to build the table among the K axis, resulting in extreme large table storage when looping spatial axes then temporal axis following the traditional GEMM, i.e., a lookup table for the whole $A[N,K]$ But if we swap the axis order from spatial first to temporal first, it will only maintain a small lookup table $[1,K]$. Therefore, T-MAC reorders the axes access to temporal axis K first and then spatial axes N and M.

Tiling. Tiling is a common technique in optimizing data locality of GEMM and reducing the memory requirements by reusing data on on-chip memory. Assume the GEMM $C[N,M] = A[N,K] \times W[M,K]$ is processed with tile $A[N_{tn}, K_{tk}]$ and $W[M_{tm}, K_{tk}]$, processing a tile requires $N_{tn} * K_{tk} + M_{tm} * K_{tk}$ data loading from DRAM to processor's on-chip memory instead of $N_{tn} * M_{tm} * K_{tk}$ data loading. In traditional GEMM, the tile size N_{tn} and M_{tm} have equal effects on efficiency while K_{tk} does not affect the data reusing and is set to align the memory transaction.

However, in LUT-based GEMM, the activation $A[N,K]$ should be processed to build the lookup table, while the weight $W[M,K]$ can share the same pre-computed lookup table. That is to say, a larger tile size M_{tm} on M can lead to better lookup table reusing. T-MAC will carefully consider the tiling configurations N_{tn}, M_{tm} and K_{tk} to achieve better data reusing.

Layout optimizations. Beyond optimizing the on-chip table lookups with axis reordering and tiling, we design two data layout optimizations, i.e., weight permutation and interleaving, to further improve the efficiency.

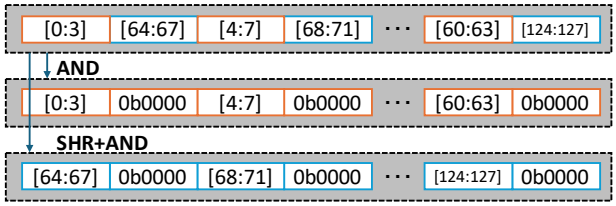

Figure 4. Interleave weights for fast unpacking.

Weight permutation for sequential memory access. DRAM requires sequential accesses to achieve higher bandwidth utilization, while accessing tiles in the tiling-based GEMM introduces random accesses because tiles of the input matrices are not sequentially stored. To solve this problem, T-MAC designs a weight permutation method that permutes the weight matrix to align the weight load with the memory transaction. After the scheduling of a GEMM is determined, T-MAC will permute the input matrices to let tiles stored sequentially instead of the whole matrices. Specifically, T-MAC flats the elements in a tile sequentially and then concatenates the flatten tiles according to the tile accessing order. Note that the weight matrix will not be modified during LLM inference, so this permutation can be done in offline. This offline permutation does not introduce cost in inference.

Weight interleaving for fast unpacking. As described in §3.1, the weight matrix is stored in packed format in memory and should be unpacked during computation. However, due to the commonly-used little-endian in modern CPUs, bytes in an integer are stored in a backward order. Therefore, using the integer instructions to unpack the weights requires additional reordering to return the correct unpacked weights. As the weight matrix will not be modified during LLM inference, T-MAC can interleave the packed weights to eliminate this reordering. Figure 4 shows an example of this interleaving that unpacking the interleaved weights can directly produce the required weights sequentially.

3.3 Reduce LUT Storage

In the realm of LUT-based method for low-bit LLM inference, the size of the lookup table is a crucial factor that impacts both the storage requirements and the access latency, especially when optimizing table look-ups with on-chip memory in §3.2. A larger table size not only demands more memory space, but also leads to slower table access speed. To address this challenge, we introduce two optimization strategies: *mirror consolidation* and *table quantization*. As illustrated in Figure 5, mirror consolidation exploits the symmetrical properties of table values to halve the length of the table, while table quantization applies quantization techniques to the table values themselves to reduce the width of the table. Combined, these methods enable a significant reduction in

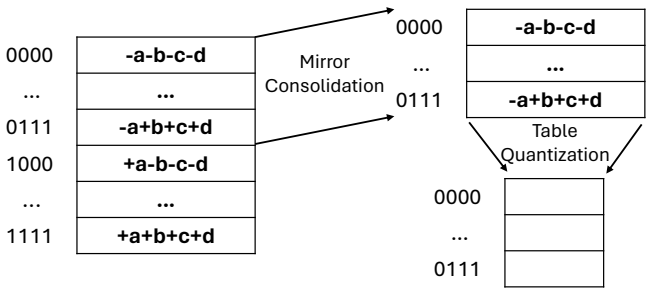

Figure 5. Reduce LUT storage with mirror consolidation and table quantization. Mirror consolidation halves the table length. Table quantization reduces the table width.

the storage footprint of the lookup table (up to a quarter of its original size) without accuracy loss in the LLM inference.

Mirror Consolidation. The inherent symmetrical properties of table values in the context of lookup tables for LLM inference present a unique opportunity for optimization. Each positive value within the table is naturally paired with its negative counterpart, reflecting a mirror image across the zero value. Leveraging this symmetry, our proposed *Mirror Consolidation* technique capitalizes on the fact that only half of the table values need to be explicitly stored. The remaining half can be rapidly reconstructed by simply negating the stored values. This table compression method is lossless, preserving the model's inference accuracy entirely. Furthermore, it proves to be highly efficient, accelerating the precomputation of the lookup table, reducing the required storage, and speeding up table accesses.

Table Quantization. Table quantization operates on a principle analogous to weight and activation quantization, aiming to reduce the precision of table values for improved computational efficiency. For instance, values initially represented in 16-bit floating-point (fp16) within a lookup table can be quantized to 8-bit integer (int8) with a scaling factor. The impact of table quantization on model accuracy is negligible. Contrary to activation quantization, which has been challenging in maintaining model accuracy due to its necessity for coarse granularity and static quantization to ensure fast computation, our approach embraces finer granularity (quantizing 8 values for k=4) and dynamic quantization to minimize accuracy degradation. The findings presented in §5.6 demonstrate that our table quantization technique has an imperceptible effect on the overall model accuracy. In terms of efficiency, table quantization significantly reduces lookup table storage requirements and accelerates the lookup process.

4 Implementation

Code generation through TVM. We employ TVM [15] + LLVM [25] for code generation. This allows us to generate optimal code for GEMM of varying shapes and for different hardware, and implement common optimizations such as loop unrolling, vectorization, and constant folding. We utilize TVM Tensorize to embed hardware intrinsics into the code. AutoTVM [16] is used to automatically fine-tune the generated code for different hardware targets.

Instruction Set	Look-up	Fast Aggregation
NEON	vqtbl1q_u8	vrhaddq_u8
AVX2	_mm256_shuffle_epi8	_mm256_avg_epu8

Table 1. Hardware Intrinsics for Look-up and Aggregation

API and integration. We provide a consistent API for both C++ and Python. The GEMM functions are encapsulated into TVM PackedFunc, and the tensors can be transferred through the DLPack [1] tensor structure. This facilitates easy interoperability with other frameworks like PyTorch, Numpy, and etc. Additionally, we provide an additional wrapper for C++, where tensors can be passed through raw pointers and TVM runtime dependency is eliminated. This offers a more lightweight solution for integration into other C++ projects.

Parallelism. We utilize the TVM runtime threadpool to dynamically assign tasks to CPUs. However, when integrating T-MAC into llama.cpp, we notice an obvious conflict between the llama.cpp threadpool and the TVM threadpool. This conflict arises as threads from different threadpools compete for CPU resources, leading to a significant performance degradation. To resolve this issue, we generate C++ code using TVM, rather than directly creating library files, and then postprocess the code to remove the dependency on the TVM runtime and threadpool. The generated function will only execute computations of a single threadblock, and then we assign these threadblocks to different threads in the llama.cpp threadpool. This approach achieves better performance and compatibility with llama.cpp. The portable C++ code also offers an option for cross-platform deployment.

Efficiant table look-up by TBL/PSHUF. After loading the table into registers, we can utilize hardware intrinsics provided by ARM NEON/INTEL AVX2. Both NEON/AVX2 offer 8-bit look-up instructions. The bit width of ARM NEON is 128, which can precisely accommodate the entire table of $g = 4$. INTEL AVX2 has a bit width of 256, but the lower and upper halves are in separate 128-bit lanes. Therefore, we duplicate the table to fill the 256-bit LUT register and look up 32 different int8 weight indices with a single instruction. If the table's data type is float16, since NEON/AVX2 do not support 16-bit LUT, we split float16 into two 8-bit LUTs, one for the lower part and the other for the higher part. We can look up the lower and higher parts with two instructions and then recombine them into float16.

Determine the size of on-chip LUT. The number of on-chip LUTs is tuned for each hardware, to make sure on-chip memory can be fully utilized and LUTs won't be swapped

out for the tile. Larger on-chip memory allows more LUTs to reside, enabling more intermediate results to be aggregated before writing back. Register spill caused by excessive on-chip LUTs can lead to overhead.

g is also determined by the on-chip memory size and instruction throughput. LUT for $g = 4$ exactly fits into one register for ARM.TBL/AVX2.PSHUF. A larger g, such as $g = 5$, requires two registers and slower ARM.TBL2/AVX512.PSHUF.

Fast 8-bit aggregation. Besides table look-up, aggregation is another significant computational overhead. To speed up the aggregation, we initially aggregate the look-up results in low bit and later convert the aggregation sum to a higher precision like float16 with no accuracy loss. Moreover, we can implement fast 8-bit aggregation [13] if the table is quantized to int8. Normally, the int8 values should be converted to int16 to avoid overflow. However, int16 instructions have half the throughput of int8 aggregation. As an alternative, we can use *avg/rhadd* instructions to compute the average and minimize accuracy loss by subtracting the probabilistic bias from the final value. Notably, fast 8-bit aggregation could result in nonnegligible accuracy loss. In Table 1, we list the hardware intrinsics for look-up and aggregation on different CPU architectures.

Bit-serial linear transformation. In the decomposition mentioned in §3.1, we utilize the original values of v_i, i.e., 0 and 1. However, we can introduce a linear transformation to these values. Denote this linear transformation as $f(v_i)$ and the transformed values as $f(0) = s_0$ and $f(1) = s_1$.

To speed up precomputation and reduce quantization error, the values of s_0 and s_1 need to be chosen with care. To circumvent float-multiply instructions, we select them from the set [-1, 0, 1]. To reduce quantization error, we strive to minimize the difference between the largest and smallest entries of the lookup table (LUT). From empirical studies, we have found that $s_0 = -1$ and $s_1 = 1$ are optimal choices.

By setting the values of s_0 and s_1, we define the linear transformation f and the decomposition of W should be adjusted as follows:

$$
\begin{aligned}
f(v_i) &= \alpha_i' v_i + \beta_i', v_i = \alpha_i f(v_i) + \beta_i, \\
&\text{where } \alpha_i = \frac{1}{\alpha_i'}, \beta = -\frac{\beta_i'}{\alpha_i'} \\
W &= \sum_{i=0}^{b-1} \alpha_i 2^i W_i' + B, \\
&\text{where } W_i' = f(W_i), B = J \cdot \sum_{i=0}^{b-1} \beta_i 2^i \\
&\text{and J is a matrix of ones}
\end{aligned}
$$

Register swizzling for efficient LUT precomputation. As demonstrated in §3.1, we opt for subtraction/addition instructions over multiplication instructions to achieve better throughput. For the LUT of shape $(N, K/g, 2^g)$, the subtraction/addition can be vectorized along the K/g axis. For instance,

$$
\begin{aligned}
LUT[0, 0:8, 0] = &- A[0, 0:32:4] - A[0, 1:32:4] \\
&- A[0, 2:32:4] - A[0, 3:32:4]
\end{aligned}
$$

The indexing into A is not contiguous. By employing *LD4* in NEON and *vgatherdps* in AVX2, we can efficiently load non-contiguous data. However, when writing the non-contiguous LUT back into memory, extracting specific bytes from a SIMD register and writing to memory is highly inefficient for AVX2. To address this issue, we use register swizzling to rearrange the LUT so that it can be written back to memory contiguously. Initially, we use *vpblendvb* to blend 8-bit values from different registers into one register, followed by *vpermd* to swizzle the 32-bit values of the 256-bit, and then *vpshufb* to further shuffle the 8-bit values into the correct order. After swizzling, the LUT can be written back to memory in a contiguous manner.

Device	Processor	Performance Cores	Max. Memory Bandwidth (GB/s)
M2-Ultra	Apple M2-Ultra	16	819.2
Raspberry Pi 5	ARM Cortex-A76	4	17.1
Jetson AGX Orin	ARM Cortex-A78AE	12	204.8
Surface Book 3	Intel Core i5-1035G7	4	58.2

Table 2. Hardware device specification.

5 Evaluation

We evaluate T-MAC with real-world, low-bit LLMs, specifically Llama and BitNet, across four distinct edge devices. Our benchmarking efforts are aimed at a direct comparison with the existing state-of-the-art *llama.cpp* implementation. We summarize our key findings as follows:

- The mpGEMV and mpGEMM kernels of T-MAC show a marked performance gain, significantly outperforming the state-of-the-art dequantization-based kernels.
- T-MAC enables an end-to-end model inference throughput improvement of 2-4x, while concurrently reducing the energy consumption by 60%-70% relative to the original *llama.cpp* implementation.
- Remarkably, in many cases, T-MAC not only matches but even exceeds GPU performance, indicating a new milestone for LLM deployment efficiency on devices.

5.1 Evaluation Setup

Hardware devices. As shown in Table 2, we evaluate T-MAC across four distinct edge devices. These devices range from high-performance ones like M2-Ultra to less powerful ones like Raspberry Pi. The CPUs tested encompass Intel Core, Apple Silicon, and Cortex series. The operating systems include OSX, Linux, and Windows. This evaluation guarantees T-MAC's cross-platform compatibility and consistent

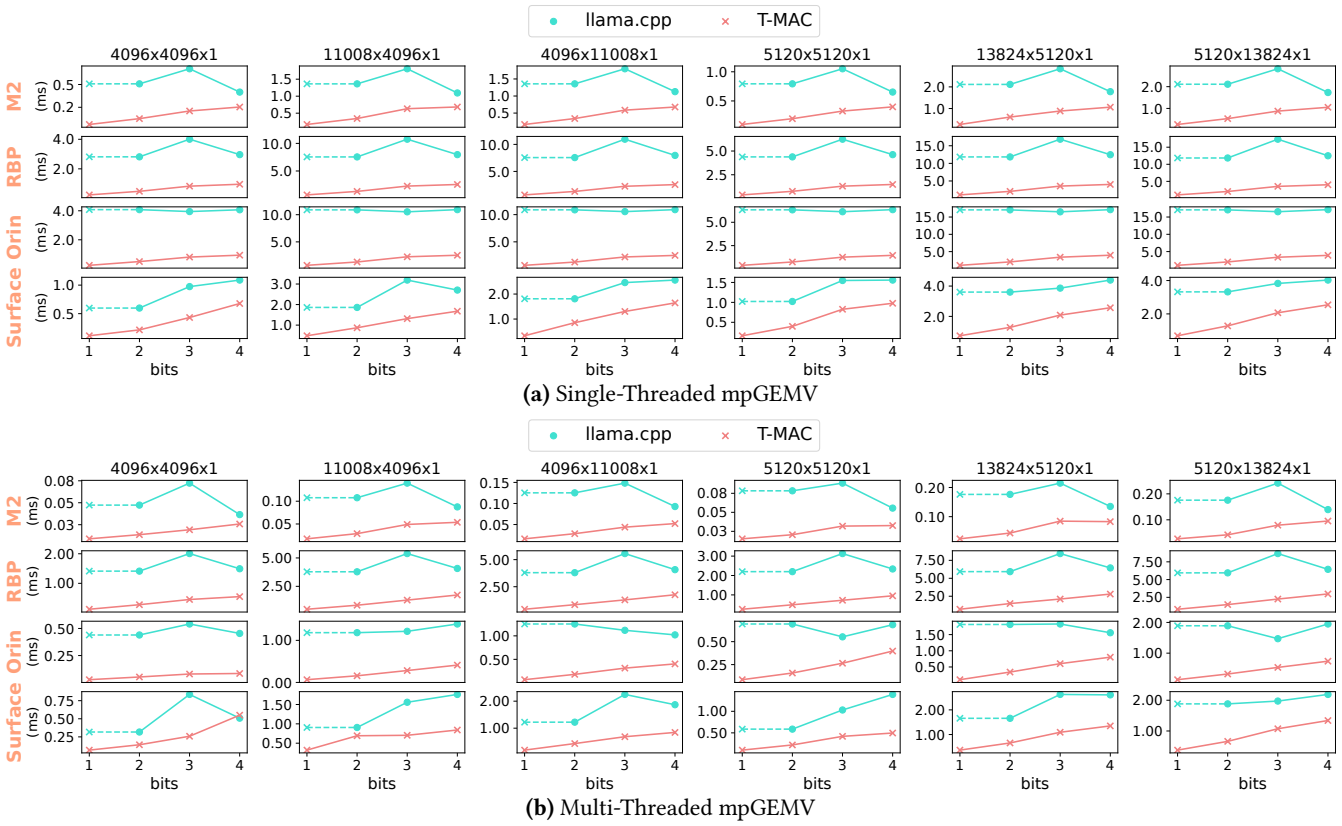

Figure 6. mpGEMV performance benchmark at 1/2/3/4 bits with single-thread and multi-thread. Matrix shapes are from Llama-2-7b and Llama-2-13b. The 1-bit kernel performance of llama.cpp is deduced from its 2-bit kernel and marked with dashed lines.

performance across different instruction sets and various edge deployment scenarios.

Kernels and models. To evaluate the performance of T-MAC, we conduct extensive benchmarks using real-word low-bit LLMs and scenarios. For the kernel performance benchmark, we select matrix shapes derived from the Llama-2-7B and Llama-2-13B models, ensuring our evaluation reflects the practical demands. To conduct an end-to-end throughput test, we employed actual quantized models to demonstrate the practical efficacy of T-MAC across different bit-width configurations. Specifically, we employ 4-bit,3-bit,2-bit and 1-bit quantized Llama models, and also 1-bit and 1.58bit BitNet models that are trained from scratch. Ternary weights in 1.58bit BitNet are interpreted as 2-bit and decomposed into two 1-bit matrices. The 4-bit Llama models are from GPTQ [21]. The 3-bit and 2-bit Llama models are from BitDistiller [19]. The 1-bit Llama models are from OneBit [36].

Baselines. We compared the performance of T-MAC with *llama.cpp* (version b2794, realesed on May 2024), a state-of-the-art implementation for LLM deployment on edge devices. We chose *llama.cpp* as the baseline for several compelling reasons. Firstly, *llama.cpp* represents the cutting-edge in LLM deployment on edge devices, featuring highly optimized kernel implementations tailored to each hardware platform. Its versatility and robust performance make it an ideal benchmark for assessing the efficacy of new methodologies. Additionally, *llama.cpp* is implemented in plain C/C++ without any dependencies, ensuring maximum compatibility and efficiency across diverse hardware configurations. For kernel performance benchmarks, we utilized the optimized kernels provided by *llama.cpp* as the baselines on the respective hardware devices. In our end-to-end throughput evaluations, we integrate the LUT-based kernels from T-MAC to *llama.cpp* and compare it with original *llama.cpp*.

We also compared the performance of T-MAC with *llama.cpp (BLAS)*. *llama.cpp* uses *Accelerate* on M2-Ultra and *OpenBLAS* on the other platforms. *llama.cpp (BLAS)* is slower for mpGEMV but faster for mpGEMM compared to *llama.cpp*'s highly optimized mixed-precision implementation. Therefore, T-MAC is measured against *BLAS* only for mpGEMM.

Measurement approach. We perform both *kernel-level* and *model-level* measurement. To obtain precise and consistent kernel-level latency on CPU, we first perform a warmup of 10 iterations, followed by 100 runs to calculate an average. The warmup on M2-Ultra differs slightly from the others, requiring at least 1 second to maximize performance. To perform model-level latency, we integrate T-MAC into llama.cpp. We repeatedly generate 64 tokens for 20 iterations to evaluate token generation throughput.

5.2 mpGEMV/mpGEMM Performance Benchmark

We evaluate the kernels in Llama-2-7B/13B across all four devices. As illustrated in Figure 6 with the bits decrease from 4-bit to 2-bit, llama.cpp fails to gain any additional speedup, and even experiences a 15% slowdown at 3-bit compared to 4-bit due to decoding overhead. Therefore, we can infer that the 1-bit llama.cpp performance would be similar to 2-bit, even though llama.cpp does not provide a 1-bit implementation. In contrast, T-MAC achieves linear speedup with bit reduction. For single-threaded GEMV, T-MAC achieves maximum speedups of 11.2x, 5.8x, 4.7x, and 3.1x respectively for 1/2/3/4 bits. For multi-threaded mpGEMV, T-MAC's performance is primarily constrained by memory bandwidth, but T-MAC can still achieve significant speedup due to efficient memory access. For example, with 2-bit, T-MAC achieves 4.0x, 4.0x, 5.31 and 2.5x on all four devices respectively.

We evaluate mpGEMM with a sequence length of 256 for multi-threading in Figure 7. *llama.cpp* uses BLAS for mpGEMM. T-MAC still achieves significant speedup on RBP, Orin, and Surface for 2-bit with maximum speedups of 4.0x, 5.3x, and 5.3x respectively. M2-Ultra is an exception, as the Apple Silicon CPUs are equipped with a powerful AMX coprocessor to handle GEMM operations. However, T-MAC still achieves a maximum 2.0× speedup for 1-bit in this case.

T-MAC demonstrates a significant advantage at 3-bit precision. This can be attributed to the inefficiencies in current techniques for handling 3-bit weights. Weights decoding is typically executed using SHIFT and AND instructions, which require weights being aligned with the width of bytes. Since 8 is indivisible by 3, this decoding process is notably inefficient. *llama.cpp* attempts to optimize it by separately packing 2 bits and the remaining 1 bit, but it still results in significant overhead. In contrast, T-MAC avoids this problem by individually computing the results of each bit.

5.3 End-to-End Inference Throughput

After integrating into llama.cpp, we compare the end-to-end token generation throughput of llama.cpp with T-MAC. We employ 2-bit to execute BitNet. As depicted in Figure 8, under single-threading on Raspberry Pi 5, T-MAC achieves speedups of 2.8x, 6.7x, and 5.8x for the three models respectively. Under multi-threading, due to memory constraints and operators other than mpGEMV/mpGEMM, the speedup is less pronounced. However, we still observe speedups of 1.1x, 2.3x, and 1.7x on M2-Ultra. T-MAC can reach a peak of 71 tokens/sec on the powerful M2-Ultra and 11 tokens/sec on the less powerful Raspberry Pi 5, indicating promising real-world edge deployment.

5.4 Power and Energy Consumption

In addition to computational efficiency, energy efficiency is equally critical, especially for edge devices that rely on battery power. To evaluate the power and energy consumption of T-MAC relative to *llama.cpp*, we conducted experiments using multi-threaded implementations on the M2 Ultra device. We selected three models for our analysis: Llama-2-7B-4bit, Llama-2-7B-2bit, and BitNet-3B. The power usage is measured using *powermetrics* on OSX, which can record the average power usage over a specified sample interval. We set this interval to 500 milliseconds, continuously generate tokens for a minimum of 120 seconds, and compute the integral of power over time to determine the total energy consumption.

The results, as depicted in Figure 9, indicate a significant reduction in power consumption when using the LUT-based kernels in T-MAC. For the models Llama-2-7B-4bit, Llama-2-7B-2bit, and BitNet-3B, T-MAC demonstrates a power consumption reduction of 10.3%, 10.3%, and 17.3%, respectively. These reductions in power consumption, combined with the latency gains offered by T-MAC, lead to a substantial decrease in total energy consumption. Specifically, T-MAC reduces energy consumption by 20.6%, 61.2%, and 51.3% for each model, respectively.

5.5 Optimization Breakdown

To evaluate the effectiveness of the optimizations in §3, we break down our optimization strategies. Most of these optimizations yield greater benefits for single-threading, but tiling requires multi-threading to be effective, hence this evaluation is conducted using multi-threading. While most optimizations are orthogonal, some are dependent on others, for instance, the permutation requires tiling to be completed first. Consequently, we begin with a basic implementation, *TM-base*, and progressively apply optimizations.

The *TM-base* version utilizes hardware intrinsics to speed up table lookup, but does not implement any memory access optimization. As depicted in Figure 10, its performance is at most 17% slower compared to the llama.cpp baseline. After implementing *table quantization*, the performance becomes competitive with llama.cpp. The *tiling* optimization further yields a maximum speedup of 1.45x. By rearranging the data layout into contiguous memory for each tiling, *permutation* contributes an additional 1.39x speedup. *Tuning* does not appear to be very effective in the figure, as the default tiling configurations already align well with M2-Ultra registers and caches, but for different devices, tuning should assist in finding a better configuration.

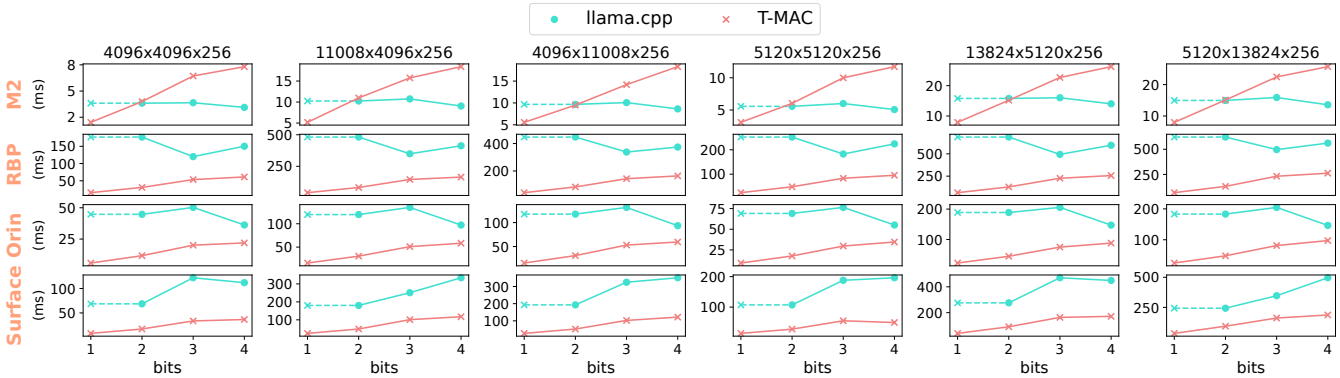

Figure 7. mpGEMM performance benchmark at 1/2/3/4 bits with multi-thread. Matrix shapes are from Llama-2-7b and Llama-2-13b with an input sequence length of 256. The 1-bit kernel performance of llama.cpp is deduced from its 2-bit kernel and marked with dashed lines.

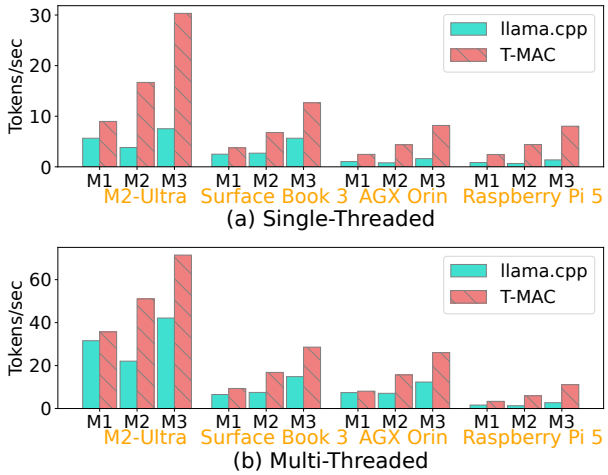

Figure 8. End-to-end token generation throughput by integrating T-MAC kernels into llama.cpp. M1, M2 and M3 stands for Llama-2-7B-4bit, Llama-2-7B-2bit and BitNet-3B.

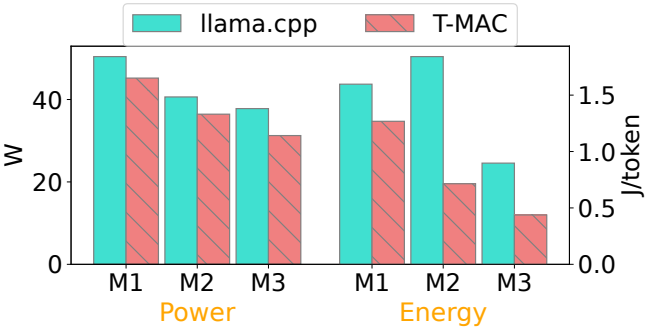

Figure 9. Power and energy consumption for multi-threaded inference on M2-Ultra. M1, M2 and M3 stands for Llama-2-7B-4bit, Llama-2-7B-2bit and BitNet-3B respectively.

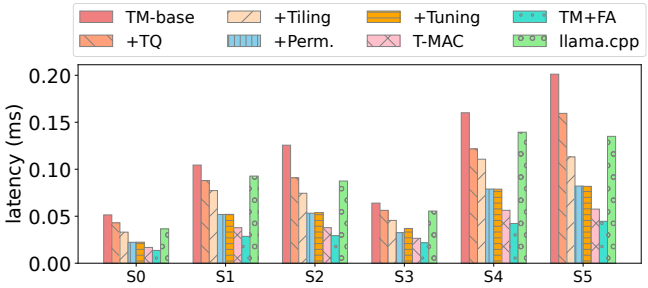

Figure 10. Multi-threaded performance of Llama-2-7B/13B GEMV kernels on M2-Ultra by applying T-MAC optimizations step-by-step. S0-S5: different shapes as shown in Figure 6. TM: T-MAC, TQ: Table Quantization, Perm.: Permutation, IL: Interleaving, FA: Fast Aggregation.

Upon applying *weights interleaving*, we obtain T-MAC. Interleaving eliminates most of the unpacking overhead, achieving a 1.42x speedup. The aggressive *fast aggregation* can make T-MAC up to 1.29x faster, but it could lead to non-negligible accuracy loss, so we offer it as an optional optimization.

5.6 Error Analysis

There are two sources of error compared to conventional mpGEMM implementation: (a) *table quantization*, which is an algorithmic approximation included in our method, and (b) *fast aggregation*, whose error is introduced during the instruction execution within the fixed CPU architecture. We evaluate the impact of these two error sources at both kernel-level and model-level.

Kernel-level Evaluation. We use the unquantized $W_{FP16}A_{FP16}$ GEMV as the benchmark. The weights and activation of the GEMV are randomly generated FP16 values following a *Gaussian Distribution*, which are then quantized to 4-bit for execution by llama.cpp and T-MAC. The Normalized Mean Squared Error (NMSE) is then computed between the ground truth and the mpGEMV outputs. As shown in

MxKxN	llama.cpp	T-MAC	T-MAC (+FA)
4096x4096x1	3.33e-03	3.35e-03	8.09e-03
11008x4096x1	3.44e-03	3.46e-03	8.27e-03
4096x11008x1	4.13e-03	4.15e-03	8.45e-03

Table 3. NMSE error relative to un-quantized ($W_{FP16}A_{FP16}$) GEMV kernel.

Framework	Throughput Tokens/sec ↑	WikiText2 PPL ↓	lambada_openai PPL ↓	WinoGrande Acc. ↑
Un-quantized	3.79	5.80	12.65	71.0
llama.cpp	5.65	5.96	12.95	70.8
T-MAC	7.34	5.96	12.95	70.8
T-MAC (+FA)	8.97	6.38	13.99	67.8

Table 4. End-to-end throughput and model quality of Llama-2-7B-4bit on M2-Ultra with single-thread. T-MAC improves throughput by 1.3× compared to llama.cpp with the same model quality. Fast Aggregation (FA) can further improve the throughput gain to 1.6×, but the model quality will drop because of the numerical error of current CPU instructions.

Table. 3, the NMSE difference between llama.cpp and T-MAC is negligible, indicating that the table quantization error is minimal. However, after applying *fast aggregation*, the NMSE increases to 2.5×.

Model-level Evaluation. To examine the impact of these errors on real-world models, we chose Llama-2-7B for testing. The models are the GGUF model converted from official Llama-2-7B weights for the un-quantized ground truth and the original llama-2-7b.Q4_0.gguf model [4] released with llama.cpp for mpGEMM. After integrating T-MAC into llama.cpp, we conduct the evaluation through the *perplexity* [6] tool provided by llama.cpp. The evaluation is performed on three different tasks: *WikiText-2* [28] and *lambada_openai* [29, 31] for perplexity (the lower the better), and *WinoGrande* [11] for question answering accuracy (the higher the better. As shown in Table 4, on all of the three tasks, T-MAC delivers the same results compared to llama.cpp, suggesting that the error introduced by T-MAC is negligible for real-world models. After toggling on the *fast aggregation*, the perplexity increases by 0.4 and 1.0 respectively and the accuracy drops by 0.3%.

In summary, T-MAC introduces negligible error to model inference while offering significant speedup. The *fast aggregation* can further enhance performance, but at the cost of model quality. We offer this as an option for users in scenarios that prioritize real-time performance and are less sensitive to accuracy. Without *fast aggregation*, T-MAC can still achieve substantial gain according to Figure 10. In the future, we anticipate the error introduced by fast aggregation can be mitigated with straightforward optimizations of the CPU micro-architecture.

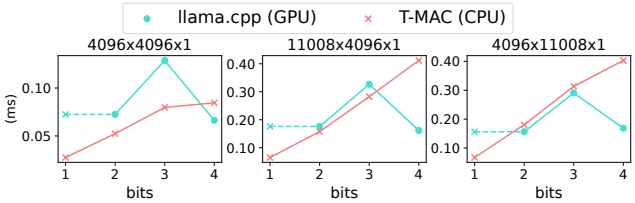

Figure 11. mpGEMV kernels performance of T-MAC (CPU) and llama.cpp (GPU) on NVIDIA Jetson AGX Orin.

Framework	Throughput Tokens/sec	Power W	Energy J/token
llama.cpp (CPU)	7.08	15.0	2.12
llama.cpp (GPU)	**20.03**	30.8	1.54
T-MAC (CPU)	15.62	**10.4**	**0.66**

Table 5. Llama-2-7B-2bit end-to-end inference throughput, power and energy comparisons on NVIDIA Jetson AGX Orin.

Device	Total CPU Cores	Used Cores	GPU	NPU
Surface Laptop 7	12 Oryon, 3.8 GHz	4	Adreno X1-85 4.6 TFLOPS	Hexagon 45 TOPS
OnePlus 12	1 Cortex-X4, 3.2 GHz 5 Cortex-A720, 3.0 GHz 2 Cortex-A520, 2.3 GHz	1 3 0	Adreno 750 4.6 TFLOPS	Hexagon 15 TOPS
Jetson Orin NX	8 Cortex A78AE, 2.0 GHz	6	Ampere GA10B 50 TOPS	-

Table 6. Detailed CPU/GPU/NPU specifications of the tested platforms. "Used Cores" refers to the number of CPU cores we used to fully utilize the memory bandwidth.

5.7 Compared with GPU/NPU

GPUs are widely used in LLM deployments. We compare T-MAC on CPU with llama.cpp on GPU to illustrate the efficiency of T-MAC. llama.cpp is the state-of-the-art LLM inference framework for both CPU and GPU on edge devices.

Figure 11 shows the mpGEMV kernel performance comparsion of T-MAC (CPU) and llama.cpp (GPU) on NVIDIA Jetson AGX Orin, a platform with ARM CPU and NVIDIA CUDA GPU. The kernel configurations are all from Llama-2-7B. T-MAC significantly outperforms GPU on W1A16 on all cases, while achieves comparable performance on W2A16 and W3A16. Although GPU performs better on higher bits and larger shape due to its powerful parallel computing capacity, this evaluation still shows huge potential of CPU-based LLM deployments on edge devices.

Table 5 shows the end-to-end comparison of the Llama-2-7B-2bit model on NVIDIA Jetson AGX Orin. Without T-MAC, CPU only performs better than GPU in power, however, the energy consumption is still worse than GPU due to lower throughput. Compared to llama.cpp on CPU, T-MAC not only improves the throughput to 2.2×, but also reduces the power to 69%, resulting in 3.2× energy efficiency. Compared to llama.cpp on GPU, although T-MAC only achieves 78% throughput, T-MAC only needs 34% power, resulting in 2.3×

Device	Model	T-MAC (CPU)	llama.cpp (CPU)	llama.cpp (GPU)	NPU
Surface Laptop 7	Llama-2-7B-4bit	**21.63**	10.64	-	10.40
	Llama-2-7B-2bit	**31.83**	9.39		10.40*
OnePlus 12	Llama-2-7B-4bit	10.19	8.24	1.60	**11.30**
	Llama-2-7B-2bit	**16.62**	6.95	1.72	11.30*
Jetson Orin NX	Llama-2-7B-4bit	7.53	3.97	**14.76**	-
	Llama-2-7B-2bit	**11.41**	3.20	7.94	

Table 7. Token generation tokens/s of T-MAC vs GPU/NPU for Llama-2-7B-4bit/2bit on three devices. The 2-bit performance of NPUs is deduced from 4-bit and marked with "*".

energy efficiency. Note that Figure 11 shows T-MAC outperforms the GPU on the mpGEMV kernels. The reason why the throughput of T-MAC is still lower than that of GPU is due to the performance of kernels except mpGEMVs in llama.cpp on CPU.

In addition to its power efficiency, T-MAC also demonstrates superior performance over GPU/NPUs across widely used platforms. We further evaluate T-MAC on three devices: Surface Laptop 7, OnePlus 12 and Jetson Orin NX. The full specifications of equipped CPU/GPU/NPUs are detailed in Table 6. We utilize the minimum number of CPU cores that can fully leverage the memory bandwidth and nearly achieve optimal performance. GPU evaluations use the llama.cpp CUDA backend for the NVIDIA GPU and OpenCL backend for the Qualcomm GPU. The performance of NPUs are sourced from official data released by Qualcomm via Qualcomm AI Hub [10].

With T-MAC, CPUs can achieve much higher computation throughput and fully exploit the memory bandwidth. GPU/NPUs, which are also memory-bound for GEMV during token generation, share unified memory with CPUs on most edge devices. As demonstrated in Table 7, T-MAC achieves significant speedup for Llama-2-7B-2bit on all three devices. Specifically, T-MAC achieves 3× speedup on Surface Laptop 7 over the NPU with only 4 out of the total 12 CPU cores, 1.5× speedup over the NPU on OnePlus 12, and 1.4× speedup over the Ampere GPU on the Jetson Orin NX. Even for Llama-2-7B-4bit, T-MAC maintains 2.1× speedup on Surface Laptop 7. Notably, on OnePlus 12, T-MAC demonstrates a substantial speedup of 6.4× and 9.7× over the Adreno GPU for 4-bit and 2-bit respectively.

In summary, T-MAC, leveraging widely available CPUs, delivers a notable performance advantage over GPUs, and even NPUs specially designed for AI workloads. This makes T-MAC a practical solution for LLM deployment on edge devices.

6 Related Works

LLM Quantization Algorithm. LLM quantization has emerged as a crucial technique for the efficient deployment of LLMs in resource-constrained environments. A segment of the research has been dedicated to the dual quantization of both weights and activations. *LLM.int8()* [17] isolates outlier feature dimensions to 16-bit computations while processing majority dimensions in efficient 8-bit computations. *SmoothQuant* [35] migrates the quantization difficulty from activations to weights with a mathematically equivalent transformation to enable an INT8 quantization of both weights and activations. Advancements in the field have led to a refined focus on the singular quantization of model weights, as weight storage accounts for the majority of memory footprint. Specific algorithms such as *GPTQ* [21] and *AWQ* [26] have demonstrated the feasibility of quantizing LLMs to just 4 bits using post-training quantization techniques. Furthermore, *BitDistiller* [19] pushes the boundary to 2 bits by leveraging quantization-aware training (QAT) and self-distillation. Meanwhile, *BitNet* [34] takes an even more ambitious route by training 1-bit LLMs from scratch.

LLM Inference System. The significance of LLMs has spurred the development of various LLM inference systems, tailored to different platforms and optimized for specific goals. *vLLM* [24] is a high-throughput and memory-efficient inference engine designed for LLMs, which excels in large batch processing. *llama.cpp* [5] stands out with its plain C/C++ implementation, free of external dependencies, which delivers superior performance on edge computing devices. *TensorRT-LLM* [9] incorporates a suite of state-of-the-art optimizations specifically for NVIDIA GPUs. *Intel Neural Compressor* [2] provides an open-source Python library for model compression techniques, tailored to enhance the efficiency of LLMs within the Intel ecosystem. All of these inference systems share a crucial capability for supporting low-bit LLMs, which not only minimizes memory usage but also enhances computational efficiency, thereby broadening the accessibility of LLMs for diverse applications. To complement the landscape of end-to-end LLM inference systems, there are also efforts concentrated on developing highly efficient computational kernels tailored for low-bit LLMs [7, 20].

7 Conclusion

T-MAC transforms the data-type-centric multiplication to bit-wise table lookup, and provide a unified and scalable

solution for the increasingly popular mpGEMM. On the pervasively available CPUs of edge devices, T-MAC kernels achieve up to 6.6× speedup compared to llama.cpp, which makes the CPU inference speed comparable or even higher than the GPU on the same device. T-MAC thus provides a practical solution to deploy LLMs on edge devices without relying on GPU, even on a Raspberry Pi. T-MAC also opens up the broad opportunity for novel LLM hardware accelerator design based on LUT, as LUT is much more efficient in hardware implementation than multiplications.

References

[1] DLPack. https://github.com/dmlc/dlpack.

[2] Intel Neural Compressor. https://github.com/intel/neural-compressor.

[3] Introducing Copilot+ PCs. https://blogs.microsoft.com/blog/2024/05/20/introducing-copilot-pcs/#_ftn2.

[4] Llama-2-7B GGUF Models. https://huggingface.co/TheBloke/Llama-2-7B-GGUF.

[5] llama.cpp. https://github.com/ggerganov/llama.cpp.

[6] llama.cpp perplexity. https://github.com/ggerganov/llama.cpp/blob/master/examples/perplexity/perplexity.cpp.

[7] MICROSOFT BitBLAS. https://github.com/microsoft/bitblas.

[8] NVIDIA CUTLASS. https://github.com/NVIDIA/cutlass.

[9] NVIDIA TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM.

[10] Qualcomm AI Hub. https://aihub.qualcomm.com.

[11] Winogrande: An adversarial winograd schema challenge at scale. 2019.

[12] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Caio César Teodoro Mendes, Weizhu Chen, Vishrav Chaudhary, Parul Chopra, Allie Del Giorno, Gustavo de Rosa, Matthew Dixon, Ronen Eldan, Dan Iter, Amit Garg, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Jamie Huynh, Mojan Javaheripi, Xin Jin, Piero Kauffmann, Nikos Karampatziakis, Dongwoo Kim, Mahoud Khademi, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Chen Liang, Weishung Liu, Eric Lin, Zeqi Lin, Piyush Madan, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Xia Song, Masahiro Tanaka, Xin Wang, Rachel Ward, Guanhua Wang, Philipp Witte, Michael Wyatt, Can Xu, Jiahang Xu, Sonali Yadav, Fan Yang, Ziyi Yang, Donghan Yu, Chengruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. Phi-3 technical report: A highly capable language model locally on your phone, 2024.

[13] Davis Blalock and John Guttag. Multiplying matrices without multiplying. In *International Conference on Machine Learning*, pages 992–1004. PMLR, 2021.

[14] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. Quip: 2-bit quantization of large language models with guarantees, 2024.

[15] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, OSDI'18, page 579–594, USA, 2018. USENIX Association.

[16] Tianqi Chen, Lianmin Zheng, Eddie Q. Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. *CoRR*, abs/1805.08166, 2018.

[17] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.

[18] Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning*, pages 7750–7774. PMLR, 2023.

[19] Dayou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. Bitdistiller: Unleashing the potential of sub-4-bit llms via self-distillation, 2024.

[20] Elias Frantar and Dan Alistarh. Marlin: a fast 4-bit inference kernel for medium batchsizes. https://github.com/IST-DASLab/marlin, 2024.

[21] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[22] Darshan C Ganji, Saad Ashfaq, Ehsan Saboori, Sudhakar Sah, Saptarshi Mitra, Mohammadhossein Askarihemmat, Alexander Hoffman, Ahmed Hassanien, and Mathieu Leonardon. Deepgemm: Accelerated ultra low-precision inference on cpu architectures using lookup tables. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4655–4663, 2023.

[23] Dibakar Gope, Jesse Beu, and Matthew Mattina. High throughput matrix-matrix multiplication between asymmetric bit-width operands, 2020.

[24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[25] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, CGO '04, page 75, USA, 2004. IEEE Computer Society.

[26] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[27] Saeed Maleki. Look-up mai gemm: Increasing ai gemms performance by nearly 2.5 x via msgemm. *arXiv preprint arXiv:2310.06178*, 2023.

[28] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[29] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics.

[30] Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. Lut-gemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models, 2023.

[31] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[32] Xiaohu Tang, Yang Wang, Ting Cao, Li Lyna Zhang, Qi Chen, Deng Cai, Yunxin Liu, and Mao Yang. Lut-nn: Empower efficient neural network inference with centroid learning and table lookup. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–15, 2023.

[33] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M

Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[34] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

[35] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.

[36] Yuzhuang Xu, Xu Han, Zonghan Yang, Shuo Wang, Qingfu Zhu, Zhiyuan Liu, Weidong Liu, and Wanxiang Che. Onebit: Towards extremely low-bit large language models. *arXiv preprint arXiv:2402.11295*, 2024.

[37] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024.